

STREAM LAYOUT / PROFILES 0 + 1

Common stream

RGI stores one lossless RGBA8 image. Extension: `.rgi`. MIME type: `image/rgi`. All multibyte integers are unsigned and **little-endian**. Pixels run left to right, then top to bottom; each output pixel is four bytes: red, green, blue, alpha. All channel values are preserved, including RGB where alpha is zero. The stream carries no colorspace, gamma, orientation, animation, metadata, or alpha-premultiplication field. Applications determine how to interpret RGBA.

14-byte header | profile-specific payload | 8-byte end marker

File offset	Bytes	Field
0	4	Magic: ASCII <code>rgif</code> = hex <code>72 67 69 66</code>
4	4	Width, 1..16384
8	4	Height, 1..16384
12	1	Channel count: 4
13	1	Payload profile: 0, 1, or 2; others reserved

Every payload must emit exactly $N = \text{width} * \text{height}$ pixels. Tokens are byte aligned and may cross row boundaries. In the diagrams, bytes appear in file order and opcode bits are written most significant first. `u16le` means two bytes, low byte first; `u8` is one byte.

Shared state for profiles 0 and 1

Start with previous pixel $P = (0, 0, 0, 255)$ and 64 cache entries equal to $(0, 0, 0, 0)$. A pixel's cache slot is:

$$h(r,g,b,a) = (3r + 5g + 7b + 11a) \& 63$$

RGB, RGBA, DIFF, and LUMA set P , emit it, and assign $\text{cache}[h(P)] = P$. INDEX sets P from the selected entry and emits it without changing the cache. RUN retains P . **A run starting at output position zero inserts the initial P into its hash slot; runs starting later leave the cache unchanged.** All component additions wrap modulo 256.

Profile 0 - RGB deltas and cache

Recognize full-byte tags FE and FF before interpreting the top two bits.

Bytes / bit fields	Operation
00iiiiii	INDEX: emit $\text{cache}[i]$, $i = 0..63$
01rrggbb	DIFF: add $r-2$, $g-2$, $b-2$ to RGB; retain alpha
10gggggg rrrrbbbb	LUMA: $dg = g-32$, $dr = dg+r-8$, $db = dg+b-8$; add to RGB, retain alpha
11nnnnnn, hex C0..FD	RUN: emit $n+1$ copies of P , count $1..62$
FE r:u8 g:u8 b:u8	RGB: replace RGB; retain alpha
FF r:u8 g:u8 b:u8 a:u8	RGBA: replace all channels

Profile 1 - Literal spans and back references

Keep profile-0 INDEX, DIFF, LUMA, RGB, and RGBA. Short RUN uses only C0..FA, counts 1..59. Recognize the following full-byte tags before short RUN:

Bytes / fields	Operation
FB n:u8 offset:u16le	COPY: copy $n+4$ earlier output pixels, count $4..259$
FC count:u8 rgba[4*count]	RAWSPAN: emit 1..255 literal RGBA pixels
FD count:u16le	LONG_RUN: emit 1..65535 copies of P

COPY and RAWSPAN set P to their last emitted pixel and **do not update the cache**. LONG_RUN follows the same initial-run cache rule as RUN.

For every COPY, `offset` is a distance in pixels before the current output position, not a byte distance. Require $\text{count} \leq \text{offset} \leq \text{pixels_already_emitted}$ and $\text{offset} \leq 65535$. The source and destination spans never overlap; copy the existing source pixels unchanged. No token may exceed the remaining image.

PROFILE 2 / VALIDATION

Profile 2 - Palette and packed indices

The common header remains 14 bytes. At offset 14 read a u16le palette count K in 1..256. At offset 16 read exactly K RGBA entries, four bytes each. The first token is at offset 16 + 4K. Entries may repeat and retain exact RGBA values, including hidden RGB. There are **no reserved header bytes**: bytes 14 and 15 together hold K.

offset 14: K_lo K_hi | palette[0].RGBA ...
palette[K-1].RGBA | tokens

Palette count K	Bits B per index
1..2	1
3..4	2
5..16	4
17..256	8

Set previous pixel P = palette[0]. There is no hash cache in this profile. Its token meanings are independent of profiles 0/1:

Bytes / bit fields	Operation
00nnnnnn packed...	LITERAL: unpack n+1 palette indices, count 1..64
01nnnnnn	RUN: emit n+1 copies of P, count 1..64
80 n:u8 offset:u16le	COPY: copy n+4 earlier output pixels, count 4..259
81 count:u16le	LONG_RUN: emit 1..65535 copies of P
Hex 82..BF	Reserved; reject
11nnnnnn offset:u16le	Short COPY: copy n+4 earlier output pixels, count 4..67

Hex FE and FF are short COPY opcodes here, not RGB or RGBA literals. Both COPY forms obey the profile-1 offset and nonoverlap rules. Their count ranges intentionally overlap. LITERAL and COPY set P to the last pixel emitted; RUN and LONG_RUN retain P.

Literal packing

A LITERAL contains $\text{ceil}(\text{count} * B / 8)$ packed bytes after its opcode. Put the first index in the lowest B bits, then continue toward higher bits and into the next byte. Each LITERAL begins on a fresh byte. Every decoded index must be less than K; unused high bits in the final packed byte must be zero.

For B=2, indices 3, 1, 2 occupy one byte, hex 27:

bit 7 (MSB)				bit 0 (LSB)			
0	0	1	0	0	1	1	1
padding		index 2		index 1		index 0	

Exact end and checked decoding

Immediately after the token that emits pixel N-1, require these eight bytes and then end-of-file:

00 00 00 00 00 00 00 01

The marker is structural, not a checksum. Do not search for it inside the payload: decoded pixel count determines where the payload ends. No trailing bytes or unused token bytes are permitted. A container must provide any additional corruption detection it needs.

A checked decoder must reject invalid magic, profile, dimensions, or channel count; arithmetic overflow; insufficient output capacity; truncated fields; invalid palette counts or indices; nonzero literal padding bits; reserved tokens; zero RAWSPAN/LONG_RUN counts; invalid COPY offsets; any token exceeding the remaining pixels; and missing, altered, or trailing end-marker data. Input and output buffers must not overlap.

Compatibility and implementation contract

This revision documents the existing rgif stream; the document revision is not a header field and does not change the magic. Profiles 3..255 are reserved. Readers supporting only profiles 0/1 need an update to read profile 2. The reference encoder's RG_RGI_NO_PALETTE_ENCODE option restricts writing to profiles 0/1 without disabling profile-2 decoding.

Encoder choices are not normative: any valid token sequence is permitted. The reference encoder compares profile 0 with profile 1, retaining profile 0 on ties, then selects profile 2 only if its complete stream is smaller. Trusted decoding is only for complete, previously checked streams and valid buffers; the current profile-2 trusted entry point still uses checked parsing. The linked normative specification governs if this companion sheet differs.

Original companion text for rg_image. © 2026 Steven Wendel. MIT License.